

O Gargalo dos 19 Casos

Por que a camada simbólica não entrega classe positiva à camada neural — e o que a literatura publicada já resolveu, recusou ou nunca mediu.

8 de setembro de 2026 Escopo: Go · Semgrep OSS · LLM local 19 perguntas, 5 blocos

O que a pesquisa mudou

O SastBench não exige que o Semgrep tenha detectado o verdadeiro positivo.

A fonte da sua classe negativa usa o scanner *apenas* para gerar os falsos positivos. Os TPs vêm das linhas afetadas do CVE e entram no conjunto de candidatos independentemente do que a ferramenta emitiu. O gargalo de $n=19$ é uma restrição sua, não do benchmark que você cita.

Você reporta o cenário mais estrito da literatura, e só ele.

Estudos de SAST reportam a detecção em uma grade de cenários — arquivo × função, qualquer tipo × CWE casada. Casar a CWE é o mais duro de todos. Reportar os dois é convenção, não concessão.

O ruleset `p/gosec` existe, é gratuito e cobre os seus dois baldes secos.

`gosec.G304-1` (CWE-22) e `gosec.G107-1` (CWE-918) já estão no registry do Semgrep, no mesmo motor e na mesma CLI. Regras sintáticas e ruidosas — que é exatamente o insumo para o qual a sua camada neural foi construída.

Trocar a OSV.dev por outra base não resolve nada.

A OSV *agrega* o GitHub Advisory Database e o Go Vulnerability Database. Trocar por qualquer um dos dois é trocar o todo pela parte. Só duas fontes têm descoberta genuinamente diferente: MoreFixes e eyeballvul.

A sua Q3 provavelmente não tem resposta com Semgrep.

Bugs de concorrência em Go — data races, vazamento de goroutine, canal não fechado — estão fora de praticamente todo SAST. As ferramentas para isso são `go vet` e o race detector, e o segundo é dinâmico.

(a) até novembro	Cabe no prazo sem recomeçar o experimento.
(b) muda escopo	Exige alterar o escopo declarado do TCC.
(c) trabalho futuro	Justificável como seção de trabalhos futuros, não como solução adotada.
⚠ muda o objeto medido	Altera o denominador do recall, seleciona casos favoráveis, ou converte o LLM de filtro em detector. Precisa ser declarado no texto.

Bloco A

Fontes de dados

A1–A2 Outras bases com commit de fix para Go

A resposta curta é desagradável: a **OSV.dev** já é o agregador. Ela ingere o GitHub Advisory Database, o Go Vulnerability Database, PyPA, RustSec, GSD e OSS-Fuzz. Consultar o GHSA ou o `vu1n.go.dev` separadamente é consultar um subconjunto do que você já varreu — o GHSA tem pouco mais de 3.100 advisories Go contra as 9.113 entradas do seu dump.

FONTE	COBERTURA GO	COMMIT DE FIX	LICENÇA	MASSA	SOBREPOSIÇÃO C/ OSV
GitHub Advisory DB	3.100+ advisories	Às vezes; referência encorajada, não obrigatória	CC-BY 4.0	Repo git, formato OSV	Quase total — a OSV ingere
Go Vulnerability DB	Canônica para Go	Via referências	BSD-3	Sim	Total — a OSV ingere
NVD / CVE	Agnóstica	Só por referência, ruidoso	Domínio público	API	Alta, mas há CVE sem entrada OSV
CVEfixes	Pequena p/ Go (dominado por C/C++)	É o objeto do dataset	Acadêmica, script reproduzível	SQLite/Postgres	Parcial — descoberta de repo própria
MoreFixes	Não publicada por linguagem	26.617 CVEs, 31.883 commits	Ver LICENSE.txt do repo	Dump Postgres, Zenodo	Parcial — descoberta ampliada é o ponto
eyeballvul	Sem restrição; filtro por linguagem embutido	Por revisão, não por par commit-pai/fix	Open source	Sim, atualização semanal	Alta (deriva da OSV), recorte diferente
GitLab Advisory DB	Ampla	Às vezes	MIT (clone com atraso de 30 dias)	Git	Alta
Snyk DB	Ampla	Não exposto	Proprietária	API só Enterprise	—
Vul4J	Java apenas (79 vulns)	Sim, + reproduzível	Acadêmica	Sim	—

FONTE	COBERTURA GO	COMMIT DE FIX	LICENÇA	MASSA	SOBREPOSIÇÃO C/ OSV
DiverseVul / PrimeVul	C/C++	Sim	Acadêmica	Sim	—
MegaVul	Origem C/C++; versões posteriores citam Go	Sim	Acadêmica	Sim	—
CrossVul	Multi-linguagem, Go marginal	Sim	Acadêmica	Sim	—
Sobreposição alta significa que trocar não resolve. As duas linhas que importam são MoreFixes e eyeballvul, porque a diferença delas está no <i>mecanismo de descoberta</i> , não na origem do aviso.					

O que vale tentar, e por quê. O seu funil perde casos no filtro “commit de fix + repo no GitHub”. A contribuição declarada do MoreFixes é exatamente essa etapa: heurísticas de descoberta de repositório que recuperam commits de correção que a referência do NVD não aponta. Ele registra um aumento de 397% em CVEs e 480% em commits de correção sobre o trabalho anterior. Não há quebra por linguagem publicada, então o ganho em Go é *desconhecido* — mas é a única aposta do Bloco A cujo mecanismo ataca a etapa em que você perde volume.

(a) até novembro

A3 Benchmarks prontos que dispensariam a mineração

A sua desconfiança está correta. A maioria é Java, C ou Python.

BENCHMARK	GO?	NATUREZA	GRANULARIDADE DO RÓTULO
OWASP Benchmark	Não — Java (2.740 casos)	Sintético	Por caso de teste
Juliet / SARD	Não — C/C++, Java, Python, Ruby	Sintético	Pares <code>good()</code> / <code>bad()</code>
CWE-Bench-Java	Não — Java (120 validadas à mão)	Real	Por vulnerabilidade
Vul4J	Não — Java	Real + reprodutível	Por vulnerabilidade
SecurityEval	Não — Python	Geração	Por prompt
RealVuln (2026)	Ainda não — Python na v1, Go no roadmap da v2	Real, 26 repos	796 achados rotulados à mão, com 120 armadilhas de FP
eyeballvul	Sim, sem restrição de linguagem	Real, 9.631 repos	Por revisão de repositório
SastBench	Sim — 38 linguagens, Go entre elas	Real (TP) + Semgrep (FP)	Par (<code>commit</code> , <code>CWE</code>)
Dataset GoVulDect (MDPI, 2025)	Sim — 630 vulns Go rotuladas por CWE	Real, do GHSA + GitHub	Por trecho de código
A linha destacada é a que você já usa — pela metade.			

A DESCOBERTA CENTRAL DESTE RELATÓRIO

Você usa o SastBench como fonte da classe negativa e reconstrói a classe positiva por conta própria, exigindo que o Semgrep a detecte. **O SastBench não faz isso.** No artigo, o Semgrep é executado sobre a versão pré-correção apenas para gerar os negativos — “achados que não compartilham o CWE com o verdadeiro positivo daquele commit” — e os verdadeiros positivos vêm das *linhas afetadas declaradas no CVE*, mineradas do NVD segundo a metodologia do CVEfixes.

O agente recebe o repositório em um contêiner, no commit indicado, com *um grupo de CWE por vez*, e devolve `{"verdict": "true_positive" | "false_positive"}`. Nada nesse protocolo exige que o scanner tenha encontrado a vulnerabilidade. O seu $n=19$ é consequência de uma exigência que o benchmark de origem não faz.

A4 Código sintético é aceito?

É aceito, com ressalvas pesadas e bem documentadas. As críticas convergem: Juliet e o OWASP Benchmark usam casos curtos e autocontidos, com pares `good()` / `bad()` que **inflam a acurácia aparente** e permitem que ferramentas de casamento de padrão pontuem bem sem entender fluxo entre arquivos ou semântica de framework. Os datasets sintéticos representam mal a distribuição real de CWEs, e o fluxo de dados neles é linear de um jeito que produção nunca é.

O trade-off para o seu caso: código sintético resolveria o tamanho da amostra em uma tarde e destruiria a validade externa da Q2. Se usar, use como *braço controlado complementar* — “eis o desempenho no caso fácil, eis no caso real” — nunca como corpus principal, e declare a diferença. `(c) trabalho futuro` **⚠ muda o objeto medido**

Bloco B

O padrão da área

B5 Como se monta o gabarito

Não há granularidade única; há uma grade explícita. O trabalho de referência mais próximo do seu (ISSTA 2024, revisão segura de código) avalia **815 commits que introduziram vulnerabilidade** em 92 projetos, cobrindo 319 vulnerabilidades exploráveis reais, e reporta a detecção em oito cenários combinando duas dimensões:

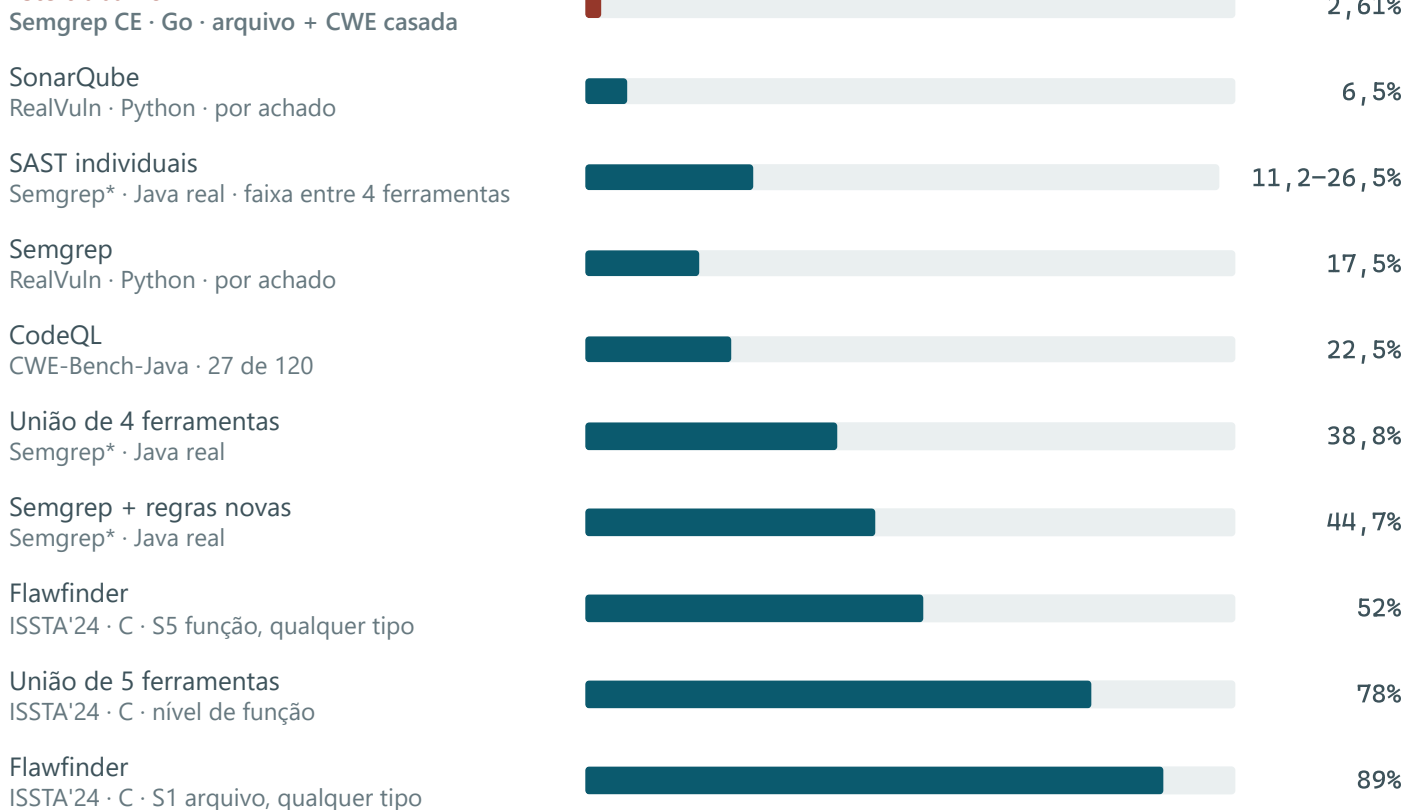
- **Granularidade** — arquivo vulnerável (S1–S4) ou função vulnerável (S5–S8).
- **Cobertura** — pelo menos um alvo atingido (`1F` / `1Fn`) ou todos (`allF` / `allFn`).
- **Casamento de tipo** — qualquer alerta (`-Any`) ou alerta da mesma CWE (`-Same`).

O SastBench, por sua vez, usa o par `(commit, CWE)` como unidade — nem linha, nem arquivo. **Seu gabarito por arquivo está alinhado com a prática publicada**, e a memória do projeto que registra essa decisão não precisa de defesa adicional. O que falta é reportar mais de um ponto da grade.

B6 2,61% é baixo demais?

É baixo, sim — mas a distância se explica por escolhas suas que são defensáveis, não por defeito de esteira.

Este trabalho



Estes números não são comparáveis entre si. Mudam a linguagem, o motor, a granularidade do acerto e a exigência de casamento de CWE. A escala serve para situar ordem de grandeza — e para mostrar que a mesma ferramenta, no mesmo corpus, varia de 52% a 89% só ao trocar “função” por “arquivo”.

O ponto de comparação mais honesto para você é o **Semgrep no RealVuln: 17,5% de recall** — mesmo motor de classe, também em código real, também sem taint entre arquivos. A diferença de 17,5% para 2,61% se explica por quatro fatores acumulados:

1. O Semgrep CE só faz taint **dentro de um arquivo**. Isso é confirmado pela documentação: análise entre arquivos para Go é recurso do Pro Engine. Os seus zero rastros de dataflow em 807 alertas são consistentes com isso.
2. Você **exige casamento de CWE** — o cenário `-Same`, o mais duro da grade.
3. Go tem menos regras que Java ou Python, e você mediu: 84 das 1.074 do `p/default`.
4. O RealVuln rotula por achado, com uma lista `acceptable_cwes` que aceita CWEs alternativas para a mesma falha. Você não tem essa folga.

Como referência de teto pela via oposta: Lipp, Banescu e Pretschner mostraram que analisadores estáticos de C **perdem entre 47% e 80%** das vulnerabilidades reais — e C é o terreno mais maduro que existe para SAST.

B7 Quando a ferramenta não detecta a CWE rotulada

Não encontrei um termo único consagrado. O que existe é uma *prática* consagrada, e ela tem duas formas:

- **Restringir o denominador ao subconjunto detectável.** É explícito na literatura de avaliação de SAST: estudos usam “o maior subconjunto do Juliet declarado detectável por todas as três ferramentas”. Você já faz o análogo disso ao filtrar a colheita pelas CWEs que o ruleset declara cobrir. **Isso é ortodoxo, não é atalho** — e a sua ressalva de circularidade sobre o grau de alcançabilidade é mais rigorosa do que o padrão da área.
- **Reportar múltiplos cenários.** A grade S1–S8 existe justamente porque o denominador é discutível. Quem publica não escolhe um; publica a matriz e deixa o leitor ver o efeito de cada critério.

A consequência prática: **a sua taxa de 2,61% é um ponto da matriz, não a matriz.** Reportar “arquivo × qualquer tipo” ao lado dela é o movimento padrão, e é o mais barato que existe — os alertas já estão no cache.

(a) até novembro

B8 Métricas para filtragem de falsos positivos

A sua escolha está alinhada, com uma lacuna fácil de fechar.

MÉTRICA	ONDE É PADRÃO	SITUAÇÃO NO SEU TRABALHO
MCC	Métrica primária do SastBench, por ser robusta a desbalanceamento	Ausente — e você usa os dados do SastBench
Precisão / Recall / F1 / F2	Auxiliares no SastBench; universais em ZeroFalse, QASecClaw, Memoir	Bloqueadas por n=19
Taxa de redução de alertas	Comum como número de manchete (ex.: 560 → 64 FPs, redução de 88,6%)	É a sua métrica principal
McNemar pareado	Recomendado por Dietterich (1998) exatamente para comparar dois classificadores num único conjunto de teste	Correto e bem aplicado
Nenhuma convenção melhor que a sua foi encontrada. Falta apenas o MCC, que custa uma função e alinha você com a fonte do seu próprio corpus.		

Bloco C

Arquitetura

C9–C10 Padrões de combinação SAST + LLM

A pergunta certa aqui não é “qual é melhor”, é “qual ataca o fato de o LLM nunca ver o caso”. A coluna do meio é a que importa.

PADRÃO	EXEMPLOS	ATACA O GARGALO?	CUSTO METODOLÓGICO
LLM como pós-filtro o seu	LLift, ZeroFalse, LLM4FPM, QASecClaw, Memoir	Não	Nenhum — preserva a pergunta de pesquisa
LLM infere especificações de taint para o motor	IRIS (ICLR 2025)	Sim — muda o que o motor emite	O LLM deixa de ser filtro puro e vira co-detector
LLM gera regras para o motor	Semgrep*, QRS	Sim	Passa a medir “as minhas regras”, não a ferramenta

PADRÃO	EXEMPLOS	ATACA O GARGALO?	CUSTO METODOLÓGICO
LLM detector independente	Braço LLM do RealVuln, eyeballvul	Sim	Deixa de ser arquitetura de duas camadas
RAG sobre base de vulnerabilidades	Vul-RAG	Parcial — melhora o julgamento, não a cobertura	Baixo; é enriquecimento declarável
Agente com ferramentas	Agentes do SastBench, QRS	Sim	Vira agente, não pipeline; muda a unidade de análise
Ensemble de motores simbólicos	ISSTA'24 (52% → 78%), Semgrep* (26,5% → 38,8%)	Sim	Comparabilidade entre rodadas anteriores
Todo padrão que ataca o gargalo cobra a mesma moeda: o LLM para de ser filtro.			

IRIS – A CITAÇÃO QUE FALTA NO SEU REFERENCIAL

O IRIS é descrito pelos próprios autores como **neuro-simbólico** — o mesmo termo do seu título — e faz exatamente o movimento que o seu gargalo pede: usa o LLM para *inferir especificações de taint* (fontes e sumidouros), injeta essas especificações em predicados do CodeQL, roda a análise, e só então aplica um segundo passo de LLM para filtro contextual. Em CWE-Bench-Java, o CodeQL sozinho detecta 27 das 120 vulnerabilidades; o IRIS com GPT-4 detecta 55, e ainda encontra 4 desconhecidas.

É a prova publicada de que a camada neural pode *alimentar* a simbólica em vez de só limpá-la. E é também a demonstração de que fazer isso custa a pergunta de pesquisa: o LLM do IRIS não é filtro.

(c) trabalho futuro

⚠ muda o objeto medido

c11 LLM detectando vulnerabilidade em Go

Não encontrei um baseline de LLM detectando vulnerabilidade em Go com recall reportado que sirva de braço de comparação direto. O que existe é adjacente:

- O RealVuln mede LLMs como scanners em código real — Claude Sonnet 4.6 com 49,8% de recall, contra 17,5% do Semgrep — mas em **Python**. Go está no roadmap da v2.
- O GoVulDect (MDPI, 2025) reporta F1 acima de 91% em Go, mas é uma **rede neural em grafo**, não LLM, e avalia sobre dataset construído pelos próprios autores.
- Estudos multi-linguagem de detecção com LLM reportam viés de contagem e degradação de recall à medida que cresce o número de vulnerabilidades por amostra.

Isso é, por si só, um achado que cabe no seu texto: **a lacuna de Go na avaliação de detecção com LLM é real e citável.**

c12 Hidratação de contexto

Há um padrão consolidado, e ele corrobora de fora o seu resultado da Q1. O ZeroFalse entrega ao LLM cinco camadas: o alerta com o identificador da regra, o **rastro completo de dataflow anotado** de fonte a sumidouro, o

código ao redor reconstruído dinamicamente, **micro-rubricas específicas da CWE** — 10 a 20 regras declarativas de conhecimento de domínio — e uma lista de verificação de adjudicação.

CORROBORAÇÃO EXTERNA DA SUA Q1

No conjunto real do ZeroFalse (OpenVuln, 58 alertas de sete projetos Java), o mesmo modelo sai de **F1 = 0,621 com prompt mínimo** para **F1 = 0,955 com prompt específico da CWE**. É a mesma direção e a mesma ordem de magnitude do seu resultado do prompt especialista contra o baseline — medida por outro grupo, em outra linguagem, com outro motor.

Duas ressalvas honestas para você. Primeira: o ZeroFalse entrega o rastro de dataflow, e você mediu que o Semgrep OSS não produz nenhum nas suas CWEs — essa camada específica você não tem como reproduzir sem trocar de motor. Segunda: a memória do projeto registra que o contexto não é o seu gargalo (prompt mediano de ~1,1k tokens), e a literatura concorda que o que faz diferença é a *estrutura* do contexto, não o volume. Estudos de ablação mostram queda ao remover contexto estrutural, conhecimento de CWE ou a explicação semântica antes do veredito — três coisas que cabem em poucos tokens.

Bloco D

Motor simbólico

D13 O que o Semgrep Pro acrescenta

Acrescenta exatamente o que falta nos seus dois baldes secos: **análise entre arquivos e interprocedural, com Go em disponibilidade geral**. A tabela de linguagens da documentação registra para Go: “Generally available · Cross-file dataflow analysis · 80+ Pro rules”.

A RESTRIÇÃO DE ORÇAMENTO TALVEZ NÃO SEJA RESTRIÇÃO

O plano gratuito do Semgrep inclui o Pro Engine, análise entre arquivos e as regras Pro para até 10 contribuidores e 10 repositórios. Para um TCC de um autor sobre repositórios públicos, isso é custo zero.

O preço não é dinheiro, é reprodutibilidade. O Pro Engine exige `semgrep login`, habilitação no painel e o download do binário via `semgrep install-semgrep-pro`. A documentação não indica operação offline. Isso colide de frente com a sua restrição de esteira offline — mitigável, porque você já tem cache simbólico: rodar uma vez, congelar o SARIF, e a partir daí a esteira volta a ser offline. Mas a colisão precisa ser dita no texto.

Números públicos: não encontrei estudo revisado por pares comparando CE e Pro. Existe uma comparação de terceiros citando 44–48% de detecção para a Community Edition contra 72–75% para o Pro em suítes de teste — *trate com desconfiança*: é material de comparação comercial, sem metodologia auditável, e provavelmente medido em suíte sintética, onde já sabemos que os números inflam. (a) até novembro

D14 Outros motores para Go

MOTOR	LICENÇA / CUSTO	TAINT EM GO	CWE-22	CWE-918	OFFLINE
Semgrep CE + <code>p/default</code>	Gratuito	Só dentro do arquivo	2 regras estreitas → 0 detecções	1 regra de taint → 1 detecção	Sim
Semgrep CE + <code>p/gosec</code>	Gratuito, registry	Sintático	<code>gosec.G304-1</code>	<code>gosec.G107-1</code>	Sim
Semgrep Pro	Grátis até 10 contribuidores	Entre arquivos, GA	Sim	Sim	Exige login
gosec	Apache 2.0	Nenhum (AST/SSA sintático)	G304	G107	Sim
CodeQL	Grátis para pesquisa acadêmica e OSS	Interprocedural	<code>go/path-injection</code>	<code>go/request-forgery</code>	Sim
staticcheck	MIT	Não	Não	Não	Sim
govulncheck	Open source	Alcançabilidade de símbolo	Incomparável	Incomparável	Precisa da vulndb
Duas linhas mudam o jogo sem mudar o motor: <code>p/gosec</code> e CodeQL. As duas são gratuitas e offline.					

GOVULNCHECK: INCOMPARÁVEL, E PIOR – CIRCULAR

Sua desconfiança está certa, e o problema é mais grave do que “trabalha por versão de dependência”. O govulncheck monta um grafo de chamadas a partir de `main.main` e reporta quando um *símbolo* sabidamente vulnerável é alcançável. Ele responde “meu programa chama código vulnerável conhecido de terceiros?”, não “este padrão de código é uma vulnerabilidade?”.

No seu corpus, o código vulnerável é o pacote, não uma dependência. Pior: o govulncheck lê a Go Vulnerability Database, que a OSV ingere, que é a origem do seu gabarito. Usá-lo como detector seria pedir que o oráculo confirmasse a si mesmo. Diga isso no texto — é uma ameaça à validade que você antecipa em vez de sofrer.

Sobre o `p/gosec`, que é a recomendação barata deste bloco. A tabela de comparação de ferramentas Go registra que o gosec “não tem análise de taint” — verdade, e é justamente por isso que ele serve. O `G304` dispara sobre caminho de arquivo construído a partir de variável, sem exigir rastro. Sobre `os.Open(filepath.Join(base, input))` — o seu exemplo canônico de CWE-22 indetectável — ele *dispara*. Vai disparar também sobre dezenas de casos seguros.

Isso não é defeito no seu desenho; é o insumo para o qual a camada neural foi construída. Uma regra ruidosa transforma 114 pares de CWE-22 de *invisíveis* em *candidatos*, e separar candidato de vulnerabilidade é literalmente a função do braço neural. E a mudança é mecanicamente pequena: `SEMGREP_CONFIG` em `src/fase1_semgrep.py:16` já parametriza o ruleset, e `src/ruleset.py` deriva a alcançabilidade do catálogo em vez de embutir a lista — a nova cobertura propaga sozinha para a colheita e para o pareamento. Falta só aceitar mais de um `--config` na invocação e na chave de cache.

D15 Combinar motores

A literatura é consistente: **ferramentas são complementares e a união eleva muito a detecção** — 52% para 78% no ISSTA'24 com cinco ferramentas em C; 26,5% para 38,8% no Semgrep* com quatro em Java; e num estudo de C/C++, de 112 CVEs, 30 só por analisadores estáticos, 33 só por fuzzers e apenas 10 por ambos.

Não quebra a sua comparação — **baseline e especialista continuam vendo exatamente o mesmo conjunto de alertas**, que é o que o McNemar pareado exige. O que quebra é a comparabilidade com as suas Rodadas 1–3, e a descrição da camada simbólica deixa de ser “Semgrep” e passa a ser “união de motores”. Se combinar, fixe a união como camada simbólica dos dois braços e reporte a contribuição por motor. (a) até novembro

D16 Escrever regras próprias

É caminho legítimo — e há precedente forte. O Semgrep* fez exatamente isso: investigou por que as ferramentas perdiam 61,2% das vulnerabilidades, identificou os padrões ausentes dos rulesets e escreveu regras novas para o Semgrep, chegando a **44,7% de detecção — acima da união de quatro ferramentas**. A análise de lacunas é a contribuição do artigo.

A ressalva é real, porém: regras suas medem as suas regras. O caminho intermediário evita o problema inteiro — **usar um ruleset publicado por terceiros** (`p/gosec`) não tem viés de autoria, não exige que você defenda nenhuma escolha de padrão, e ataca as mesmas CWEs. Escreva regras próprias só se quiser transformar a análise de lacunas em contribuição declarada, no modelo do Semgrep*. (a) até novembro

⚠ muda o objeto medido, se forem suas

AVISO NÃO SOLICITADO SOBRE A SUA Q3

A sua terceira pergunta de pesquisa foca “fraquezas estruturais de Go: concorrência, goroutines”. A literatura é direta: **bugs de concorrência em Go — data races, vazamento de goroutine, canal não fechado — não são cobertos pela maioria dos SAST**. O staticcheck não simula caminhos concorrentes nem valida risco de corrida; o `go vet` tem análise mínima de primitivas de concorrência; as ferramentas de fato são o race detector (`go test -race` , dinâmico) e pesquisa especializada como o GCatch.

Se o `p/default` não tem regra de concorrência para Go — e a sua medição de 84 regras sugere fortemente que não —, a Q3 não está com amostra pequena: está sem instrumento. Vale medir isso explicitamente e reportar como resultado, em vez de deixar a pergunta sem resposta no texto final.

Bloco E Validade e banca

E17 Ameaças que você ainda não mapeou

As quatro que você já tem — ruleset não fixado, classe positiva pequena, modelo pequeno quantizado, corpus de uma linguagem — estão corretas. Faltam estas:

AMEAÇA	POR QUÊ	GRAVIDADE
A classe negativa é aproximada por construção	O SastBench rotula como FP todo achado que não compartilha a CWE do verdadeiro positivo daquele commit, e descarta achados na mesma função. Parte desses “falsos positivos” são vulnerabilidades reais de outra CWE. Seus 98,19% são medidos contra esse rótulo.	Alta — está no método da sua fonte
Contaminação de treino	Os CVEs são públicos e anteriores ao corte do <code>qwen2.5-coder</code> . O modelo pode ter visto a correção. O eyeballvul existe precisamente para ser “à prova de futuro” contra isso.	Alta
Quantização em 4 bits	Há evidência de que a compressão em 8 bits é neutra a levemente positiva, enquanto 4 bits introduz risco significativo em comportamento de segurança de código. O seu Q4_K_M está do lado errado dessa linha.	Média — mensurável com uma rodada em Q8
Não-determinismo do braço neural	Você verificou o determinismo do Semgrep (40/40). O LLM não tem garantia equivalente sob amostragem.	Média
Validade de construto do “arquivo vulnerável”	Nem todo arquivo tocado por um commit de correção carrega a falha. Seu guarda de 8 funções mitiga, mas o resíduo é ruído no gabarito — o mesmo problema que Croft et al. documentam como qualidade de dados em datasets de vulnerabilidade.	Média — já mitigada, falta declarar
Grau de alcançabilidade derivado dentro da amostra	Circularidade que você já registrou. Mantenha o <code>--grau-minimo</code> desligado por padrão e declare.	Baixa — já tratada
A primeira linha é a que uma banca atenta encontraria lendo o artigo do SastBench.		

E18 Fixar e versionar o ruleset

Há prática estabelecida, em duas camadas:

- **Fixar o motor** — travar a imagem Docker do Semgrep numa tag ou digest específico é a recomendação explícita para resultados reproduzíveis entre execuções.
- **Vendorizar as regras** — o repositório `semgrep/semgrep-rules` é público sob a Semgrep Rules License. Clonar num commit fixo e rodar `--config ./caminho-local` elimina a variação do lado do servidor por completo. É a forma mais forte, e é a que responde à sua ameaça já mapeada.

Você está a meio caminho: `src/ruleset.py` já cacheia o catálogo por nome de configuração, justamente para não mentir quando o ruleset mudar. Falta registrar o carimbo de tempo da busca e um hash do conjunto — e, se quiser fechar a ameaça de vez, trocar `p/default` por um clone congelado. `(a) até novembro`

E19 Resultado negativo é publicável?

Sim, e há infraestrutura editorial para isso. A *Empirical Software Engineering* mantém seção especial de resultados negativos; ICSME e SANER têm trilhas dedicadas de Replicação e Resultados Negativos. O critério declarado é que o resultado seja “não óbvio ou contrarie sabedoria amplamente aceita”.

O seu se enquadra. A afirmação “a análise sintática não encontra, em código real, as fraquezas que seu próprio catálogo diz cobrir” contraria uma suposição implícita e operacionalmente importante: a de que um ruleset que declara uma CWE a detecta. E há precedentes diretos:

- **Lipp et al. (ISSTA 2022)** — analisadores de C perdem de 47% a 80% das vulnerabilidades reais, e a contribuição principal é a medição da lacuna.
- **Semgrep* (EASE 2024)** — o artigo é literalmente “por que as ferramentas não detectaram 61,2%”, e a resposta à lacuna é a contribuição.
- **ISSTA 2024** — 22% dos commits vulneráveis não são detectados por nenhuma das cinco ferramentas, com CWEs inteiras passando batido apesar de existirem regras.

Um reforço para a sua defesa: o próprio SastBench **não reporta** a proporção de CVEs reais que o Semgrep de fato sinalizou. É uma lacuna do artigo que fundou o seu corpus — e é exatamente o número que você mediu.

Fecho

O que eu faria no seu lugar

Três, em ordem de prioridade. A primeira resolve o gargalo; a segunda ataca a causa; a terceira protege o que você já tem.

1 Adotar o protocolo do SastBench para a classe positiva

Você já usa a classe negativa dele. Adote também o desenho da positiva: os verdadeiros positivos entram no conjunto de candidatos a partir das linhas afetadas do CVE, **sem exigir que o Semgrep os tenha encontrado**. Seus 690 pares reconstruídos deixam de ser matéria-prima descartada e viram classe positiva.

Isso destrava Recall, F1, MCC e taxa de falsos negativos, e responde a metade da Q2 que hoje está em branco. É citável, é publicado, e é coerente com a fonte que você já declara no TCC.

O que muda, e precisa estar escrito: o LLM deixa de ser filtro puro do Semgrep e passa a ser classificador de triagem sobre um conjunto misto — alertas do Semgrep mais localizações sabidamente vulneráveis. A Q2 muda de “quanto ruído o filtro remove” para “quanto ruído remove sem rejeitar vulnerabilidade verdadeira”, que é a pergunta que você queria responder desde o começo. Mantenha as rodadas atuais como o braço “filtro puro”: você passa a ter dois desenhos, não um substituído.

(a) até novembro

⚠ muda o objeto medido

2 Somar `p/gosec` ao ruleset e reportar dois pontos da grade de cenários

Dois movimentos baratos que atacam a causa que você mediu, sem tocar no desenho.

O ruleset: `p/gosec` traz `gosec.G304-1` para CWE-22 e `gosec.G107-1` para CWE-918 — os seus dois baldes secos — no mesmo motor, mesma CLI, offline, sem custo. São regras sintáticas e ruidosas, e é isso que as torna úteis: transformam 114 pares invisíveis em candidatos para o braço neural julgar. `SEMGREP_CONFIG` já é parâmetro e `src/ruleset.py` propaga a alcançabilidade sozinho.

Os cenários: passe a reportar a detecção em “arquivo × qualquer tipo” ao lado de “arquivo × CWE casada”. É a convenção da área desde o ISSTA'24, os alertas já estão no seu cache, e mostra o efeito do critério em vez de escondê-lo atrás de um número só. Não é afrouxamento — é a matriz que a literatura publica.

(a) até novembro

3 Reposicionar o resultado negativo como contribuição — e declarar o limite da classe negativa

2,61% é baixo mesmo para a literatura, e você tem a explicação estrutural pronta: o Semgrep CE só faz taint dentro do arquivo, e você exige casamento de CWE. Enquadre isso como medição de uma lacuna, com os precedentes de Lipp et al., Semgrep* e ISSTA'24 — inclusive o fato de que o SastBench, que fundou o seu corpus, não publica esse número.

No mesmo movimento, feche a ameaça que ninguém apontou ainda: **seus 98,19% são medidos contra uma classe negativa aproximada por construção**. Achados rotulados FP por não compartilharem a CWE do verdadeiro positivo podem ser vulnerabilidades reais de outro tipo. Declarar isso antes da banca é muito melhor do que responder a isso durante a banca.

(a) até novembro

(c) o braço LLM-detector fica p/ futuro

Nota

Sobre o que você já recusou

Nenhuma das quatro recusas foi revertida por esta pesquisa. Duas ganharam contexto novo.

Enviar a colheita para CWEs que detectam bem

Continua recusada, e a literatura reforça: é exatamente a crítica feita aos benchmarks sintéticos, que permitem à ferramenta pontuar bem sem capacidade real. Sua recusa está correta.

Afrouxar o guarda anti-refactor

Continua recusada. Croft et al. documentam o ruído de rótulo em datasets de vulnerabilidade como problema de primeira ordem, com impacto direto nos modelos a jusante. Você está do lado certo.

Trocar `p/default` por `p/golang`

Confirmada como pior. Mas note a diferença entre *trocar* e *somar*: a recomendação 2 acrescenta `p/gosec` sem remover nada, e o argumento que derrubou `p/golang` — perder 97 casos — não se aplica a uma união.

Incluir pares de extração imprecisa (`--incluir-janela`)

Continua recusada; +156 pares por +0,6 detecção é retorno nulo. A recomendação 1 resolve o mesmo problema por outra via, sem degradar a extração.

Limites

O que não consegui verificar

Registrado para você não confiar mais do que a evidência sustenta:

- **Quantas amostras Go o SastBench tem.** O artigo cita 38 linguagens e uma figura de distribuição, mas não tabula os números por linguagem no texto acessível. Confirme no dataset antes de citar.
- **A cobertura Go do MoreFixes.** O README não publica quebra por linguagem. O mecanismo é promissor, o ganho é desconhecido — teste com uma amostra antes de investir.
- **A contagem exata de regras do `p/gosec`.** A página do registry retornou 404 para mim. Os identificadores `gosec.G304-1` e `gosec.G107-1` estão confirmados por fontes secundárias; rode `semgrep --config p/gosec` e conte antes de escrever o número.
- **A comparação CE × Pro (44–48% contra 72–75%).** Vem de site de comparação comercial, sem metodologia auditável. Não cite como evidência; use só como indício.
- **Baseline de LLM detectando vulnerabilidade em Go.** Procurei e não achei com recall reportado. Se existir, está fora do que essas buscas alcançaram.
- **A licença exata do MoreFixes.** Há `LICENSE.txt` no repositório, mas não consegui ler o tipo. Verifique antes de redistribuir qualquer derivado.

Fontes

Referências consultadas

1. Feiglin & Dar. SastBench: A Benchmark for Testing Agentic SAST Triage. arXiv:2601.02941 (2026). CC BY 4.0.
2. RealVuln: Benchmarking Rule-Based, General-Purpose LLM, and Security-Specialized Scanners on Real-World Code. arXiv:2604.13764.
3. Lipp, Banescu & Pretschner. An Empirical Study on the Effectiveness of Static C Code Analyzers for Vulnerability Detection. ISSTA 2022.
4. An Empirical Study of Static Analysis Tools for Secure Code Review. ISSTA 2024 — cenários S1–S8.
5. Semgrep*: Improving the Limited Performance of SAST Tools. EASE 2024.
6. Li et al. IRIS: LLM-Assisted Static Analysis for Detecting Security Vulnerabilities. ICLR 2025.
7. ZeroFalse: Improving Precision in Static Analysis with LLMs. arXiv:2510.02534.
8. Li et al. Enhancing Static Analysis for Practical Bug Detection: An LLM-Integrated Approach (LLift). OOPSLA 2024.
9. Vul-RAG: Enhancing LLM-based Vulnerability Detection via Knowledge-level RAG. TOSEM.

10. Utilizing Precise and Complete Code Context to Guide LLM in Automatic False Positive Mitigation.
11. Croft, Babar & Kholoosi. Data Quality for Software Vulnerability Datasets. ICSE 2023.
12. Bhandari, Naseer & Moonen. CVEfixes. PROMISE 2021.
13. Akhoundali et al. MoreFixes: A Large-Scale Dataset of CVE Fix Commits. PROMISE 2024.
14. Chauvin. eyeballvul: a future-proof benchmark for vulnerability detection in the wild.
15. Go Source Code Vulnerability Detection Method Based on Graph Neural Network. Applied Sciences, 2025.
16. OSV.dev — Data sources.
17. GitHub Advisory Database (CC-BY 4.0, formato OSV).
18. GitLab Advisory Database (gemnasium-db) — clone MIT com atraso de 30 dias.
19. Semgrep — Supported languages (Go: cross-file GA).
20. Semgrep — Perform cross-file analysis.
21. Semgrep — Pricing (Pro Engine no plano gratuito, até 10 contribuidores).
22. Semgrep Registry — ruleset `gosec`.
23. gosec — Go security checker (Apache 2.0).
24. CodeQL — `go/path-injection` e `go/request-forgery`.
25. CodeQL CLI — termos (uso livre para pesquisa acadêmica).
26. govulncheck — alcançabilidade de símbolo.
27. Best SAST Tools for Go (2026) — comparação de taint por ferramenta.
28. Dietterich. Approximate Statistical Tests for Comparing Supervised Classification Learning Algorithms. Neural Computation, 1998.
29. Heckman & Williams. A systematic literature review of actionable alert identification techniques. IST, 2011.
30. Representativeness in the Benchmark for Vulnerability Analysis Tools (B-VAT). CSET 2020.
31. Foreword to the special section on negative results in software engineering. EMSE.
32. ICSME 2026 — Replication and Negative Results track.
33. Is Quantization a Deal-breaker? Empirical Insights from Large Code Models.
34. QRS: A Rule-Synthesizing Neuro-Symbolic Triad for Autonomous Vulnerability Discovery.
35. Modeling Go Concurrency: A Static Analysis Approach to Data Race Detection. Internetwork 2025.
36. QASecClaw: A Multi-Agent LLM Approach for False Positive Reduction in SAST.